
Structural Drift Repair in Decision Trees via Bayesian Model-Based Diagnosis (Supplementary Material)

Yoav Zelinger¹

Meir Kalech¹

¹The Stein Faculty of Computer Science and Information, Ben-Gurion University of the Negev, Be'er Sheva, Israel

A COMPLEXITY ANALYSIS

Let $S_D, S_{D'}$ denote the number of pre-drift and post-drift samples, F the number of features, V the number of nodes in the tree, and $\mathcal{H}$ the number of candidate diagnoses (minimal hitting sets) enumerated by BARINEL.

A.1 DIAGNOSIS COMPLEXITY

A.1.1 SFL Ranking.

STAT Computation The STAT rank (Definition 9) requires computing the pre-drift and post-drift constraint violation rates for every node in the tree. Each violation rate is obtained by passing each sample through the tree and checking constraint satisfaction at every visited node, with each sample visiting at most V nodes. Since we operate under the no-prior-data constraint, the pre-drift violation rates are pre-computed during training and stored as a scalar in each node. Thus, the STAT computation takes $O(S_{D'} \cdot V)$ **time** and $O(V)$ **space**.

Spectrum Computation Both SFL and BARINEL operate on the $SPCT_{\mathcal{T}} \in \{0, 1\}^{S_{D'} \times V}$ node-sample spectrum and the $E_{\mathcal{T}} \in \{0, 1\}^{S_{D'}}$ error vector. Constructing these requires passing each sample through the tree, recording participation at every visited node and the classification result, taking $O(S_{D'} \cdot V)$ **time and space**.

Given the spectrum, the SFL score of each node is computed from its participation column and $E_{\mathcal{T}}$ using a similarity coefficient. For a typical coefficient such as Ochiai, this requires accumulating four counters per node by comparing each entry of the participation column against the corresponding entry in $E_{\mathcal{T}}$. These counters can be accumulated during spectrum construction and stored per node, resulting in additional constant time and space per node. Thus, the SFL ranking adds only $O(V)$ **time and space**.

The final APPETITE score multiplies the two scalar ranks (SFL and STAT) for each node; the node maximizing this product is returned as the diagnosis (only the highest product needs to be stored). Altogether (STAT + Spectrum + SFL), the full APPETITE diagnosis requires:

- **Time:** $O(S_{D'} \cdot V)$.
- **Space:** $O(S_{D'} \cdot V)$.

A.1.2 BARINEL Ranking.

BARINEL reuses the same spectrum and STAT ranks (calculations in Section A.1.1). Its additional computation consists of two steps: candidate generation and candidate ranking. Since only the top-ranked diagnosis is needed for repair, candidates are scored on-the-fly and only the best is retained.

- **Candidate generation (MHS).** Following Abreu et al. [2009b], candidates are generated by computing minimal hitting sets over the classification paths of misclassified samples using the STACCATO algorithm, which takes $O(S_{D'} \cdot V)$ time and space (operates on the existing spectrum). This step produces a bounded number of candidates $\mathcal{H}$, capped at 100 in practice.
- **Candidate ranking.** For each candidate Δ_q , the posterior is computed from:
 - **Prior:** The product $\prod_{n_i \in \Delta_q} p_i \cdot \prod_{n_i \notin \Delta_q} (1 - p_i)$ using the STAT ranks as per-node fault probabilities. Besides the previous STAT cost, this calculation adds an additional $O(V)$ time and $O(1)$ space.
 - **Likelihood:** Computed via the maximum likelihood estimation procedure of Abreu et al. [2009b], which estimates per-component health parameters using gradient ascent with a constant number of iterations. This procedure takes $O(S_{D'} \cdot |\Delta_q|)$ time and $O(|\Delta_q|)$ space. However, since $|\Delta_q|$ is bounded in practice, this reduces the complexity to $O(S_{D'})$ time and $O(1)$ space per diagnosis.

Since we only retain the top diagnosis, the total ranking cost is $O(\mathcal{H} \cdot (S_{D'} + V))$ time and $O(1)$ space. However, as $\mathcal{H}$ is capped in practice, the time reduces to $O(S_{D'} + V)$.

Altogether (STAT + Spectrum + BARINEL), the full BTAD diagnosis requires:

- **Time:** $O(S_{D'} \cdot V + \mathcal{H} \cdot (S_{D'} + V))$, where $\mathcal{H}$ is bounded in practice.
- **Space:** $O(S_{D'} \cdot V)$ (dominated by the spectrum; BARINEL adds only $O(1)$).

A.2 REPAIR COMPLEXITY

A.2.1 Retraining (NEWALL/ NEWDRIFT)

Both baselines train a CART decision tree from scratch.

- **NEWALL:** Trains on $S_D \cup S_{D'}$.
 - **Time:** $O((S_D + S_{D'}) \cdot F \cdot \log(S_D + S_{D'}))$.
 - **Space:** $O((S_D + S_{D'}) \cdot F)$.
- **NEWDRIFT:** Trains on $S_{D'}$ only.
 - **Time:** $O(S_{D'} \cdot F \cdot \log S_{D'})$.
 - **Space:** $O(S_{D'} \cdot F)$.

A.2.2 APPETITE Fix (Node Adaptation)

The repair modifies a single diagnosed node: a terminal node has its label reassigned to the majority class among its reaching samples ($O(S_{D'})$); a non-terminal numeric node has its threshold shifted by the change in mean ($O(S_{D'})$, using the pre-stored pre-drift mean and the post-drift mean computed from reaching samples); a non-terminal categorical node has its constraint negated ($O(1)$).

- **Time:** $O(S_{D'})$ - identifying reaching samples dominates.
- **Space:** $O(V)$ for non-terminal numeric nodes; $O(1)$ otherwise.

A.2.3 BTAD Fix (Subtree Re-fitting)

Our repair algorithm operates in three steps:

1. **Dependency Resolution.** Ancestor–descendant pairs in $\Delta_{\mathcal{T}}$ can be resolved (Definition 15) by a single top-down pass over the tree: each node is visited once, propagating ancestry information downward. Also, nodes are replaced in $\Delta_{\mathcal{T}}$ in-place, requiring no additional space. Thus, this process results with $O(V)$ time and $O(1)$ space.

2. **Synthetic Sample Generation.** For each faulty node $n_i \in \Delta_{\mathcal{T}}$, we generate $|S_D(n_i)|$ synthetic samples, each with F features (Definition 14). Per sample per feature the cost is $O(1)$ (a Gaussian draw with clip, or a categorical draw). Generation parameters are pre-stored in each node in $O(F)$ space. Since the retained nodes are pairwise non-ancestral, their reaching sets from S_D are disjoint, giving $\sum_{n_i \in \Delta_{\mathcal{T}}} |S_D(n_i)| \leq S_D$, leading to $O(S_D \cdot F)$ time and $O(F \cdot (S_D + V))$ space.
3. **The Subtree Re-fitting.** Each node n_i is repaired by training a CART subtree on $|S_{D'}(n_i)| + |S_D(n_i)|$ samples with F features. By the same disjointness argument, the combined training data across all nodes is therefore bounded by $S_D + S_{D'}$. This leads the repair itself to take $O((S_D + S_{D'}) \cdot F \cdot \log(S_D + S_{D'}))$ time and $O((S_D + S_{D'}) \cdot F)$ space. The repair cost is at most that of full retraining (NEWALL), since the disjoint reaching sets partition the data.

Altogether (Dependency Resolution + Synthetic Sample Generation + Subtree Refitting), the full BTAD repair requires:

- **Time:** $O((S_D + S_{D'}) \cdot F \cdot \log(S_D + S_{D'}))$.
- **Space:** $O((S_D + S_{D'} + V) \cdot F)$.

A.3 SUMMARY

Table 5 summarizes the end-to-end complexity of each algorithm.

Algorithm	Time	Space
APPETITE Diagnosis	$O(S_{D'} \cdot V)$	$O(S_{D'} \cdot V)$
APPETITE Repair	$O(S_{D'})$	$O(V)$
BTAD Diagnosis	$O(S_{D'} \cdot V + \mathcal{H} \cdot (S_{D'} + V))$	$O(S_{D'} \cdot V)$
BTAD Repair	$O((S_D + S_{D'}) \cdot F \cdot \log(S_D + S_{D'}))$	$O((S_D + S_{D'} + V) \cdot F)$
Retraining (NEWALL)	$O((S_D + S_{D'}) \cdot F \cdot \log(S_D + S_{D'}))$	$O((S_D + S_{D'}) \cdot F)$
Retraining (NEWDRIFT)	$O(S_{D'} \cdot F \cdot \log S_{D'})$	$O(S_{D'} \cdot F)$

Table 5: Complexity comparison.

BTAD’s diagnosis overhead relative to APPETITE is the MHS enumeration and BARINEL ranking ($O(\mathcal{H} \cdot (S_{D'} + V))$), which is negligible in practice. Its repair phase is more expensive than APPETITE’s ($O((S_D + S_{D'}) \cdot F \cdot \log(S_D + S_{D'}))$ vs. $O(S_{D'})$), yet it is bounded by the cost of full retraining (NEWALL) and produces structurally richer output (replacement subtrees rather than single-parameter adjustments).